

AI & ASSISTED CODING

ASSIGNMENT-7.3

NAME:N.SATWIK A

BATCH:28

HT NO.:2303A51690

Task 1: Fixing Syntax Errors

PROMPT: Check this Python function and fix the syntax error.

CODE WITH ERROS:



The screenshot shows a code editor interface with a top bar containing 'Commands', '+ Code', '+ Text', and 'Run all'. On the left is a sidebar with icons for file explorer, search, and other tools. The main editor area displays a Python function definition:

```
[1] def add(a, b)
    return a+b
```

Below the code, an error message is shown:

```
... File "/tmp/ipython-input-1392594439.py", line 1
      def add(a, b)
                ^
SyntaxError: expected ':'
```

At the bottom, there is a 'Next steps:' section with a button labeled 'Explain error'.

AI FIXED CODE:

The screenshot shows a code editor with the following Python code:

```
def add(a, b)
def add(a, b):
    return a+b
```

The first line is highlighted in red, and the second line is highlighted in green. Below the code, the error message is displayed:

```
... File "/tmp/ipython-input-1392594439.py", line 1
      def add(a, b)
            ^
SyntaxError: expected ':'
```

Below the error message, there is a "Next steps:" section with a button labeled "Explain error". To the right, a panel titled "Please explain this error:" contains the following text:

SyntaxError: expected ':'

The error `SyntaxError: expected ':'` means that a colon is missing at the end of the `def` statement for the `add` function. In Python, function definitions require a colon after the parameters.

Buttons: [Accept and run](#) [Accept](#) [Cancel](#)

Below the explanation, there is a text input field with the placeholder "What can I help you build?" and a "+" button. To the right of the input field, it says "Gemini 2.5 Flash" with a dropdown arrow and a play button. At the bottom of the panel, it says "Gemini can make mistakes, so double-check it and use code with caution. [Learn more](#)".

CORRECTED CODE & OUTPUT:

The screenshot shows a code editor with the following Python code:

```
def add(a, b):
    return a+b
print(5, 9)
```

The code is highlighted in blue. Below the code, the output is displayed:

```
... 5 9
```

JUSTIFICATION:

- * Python function definitions must end with a colon (:).
 - * Missing the colon causes a `SyntaxError` during compilation.
 - * The interpreter cannot identify the start of the function body.
 - * Due to this error, the program does not execute at all.
 - * AI detects such errors by checking Python grammar rules.
 - * Adding the colon resolves the syntax issue completely.
-

TASK2: Debugging Logic Errors in Loops

PROMPT: This loop is running forever. Find the mistake and fix the loop so it stops correctly. Explain what went wrong.

CODE WITH ERRORS: LOGICAL ERROR

```
] 1 def count_down(n):  
   2     while n>=0:  
   3         print(n)  
   4         n+=1  
   5     print(count_down(9))
```

```
... 3524027  
    3524028  
    3524029  
    3524030  
    3524031  
    3524032  
    3524033  
    3524034  
    3524035  
    3524036  
    3524037  
    3524038  
    3524039  
    3524040  
    3524041  
    3524042  
    3524043  
    3524044
```

AI FIXED CODE:

9] Gemini

```
def count_down(n):  
    while n>=0:  
        print(n)  
        n+=1  
    print(count_down(9))  
+ n-=1  
+ count_down(9)
```

17422942
17422943
17422944
17422945
17422946
17422947
17422948
17422949
17422950
17422951
17422952
17422953
17422954
17422955
17422956
17422957
17422958
17422959
17422960
17422961

n correct the logical error

♦ The `count_down` function currently has a logical error that causes an infinite loop because `n` is incremented instead of decremented. I will correct this to `n -= 1`. I'll also remove the outer `print` call, as the function already prints the numbers during the countdown.

► Accept and run ✓ Accept ✕ Cancel

What can I help you build?

+ Gemini 2.5 Flash ▶

Gemini can make mistakes, so double-check it and use code with caution. [Learn more](#)

CORRECTED CODE AND OUTPUT:

[11] ✓ 0s

```
def count_down(n):  
    while n>=0:  
        print(n)  
        n-=1  
    count_down(9)
```

9
8
7
6
5
4
3
2
1
0

JUSTIFICATION:

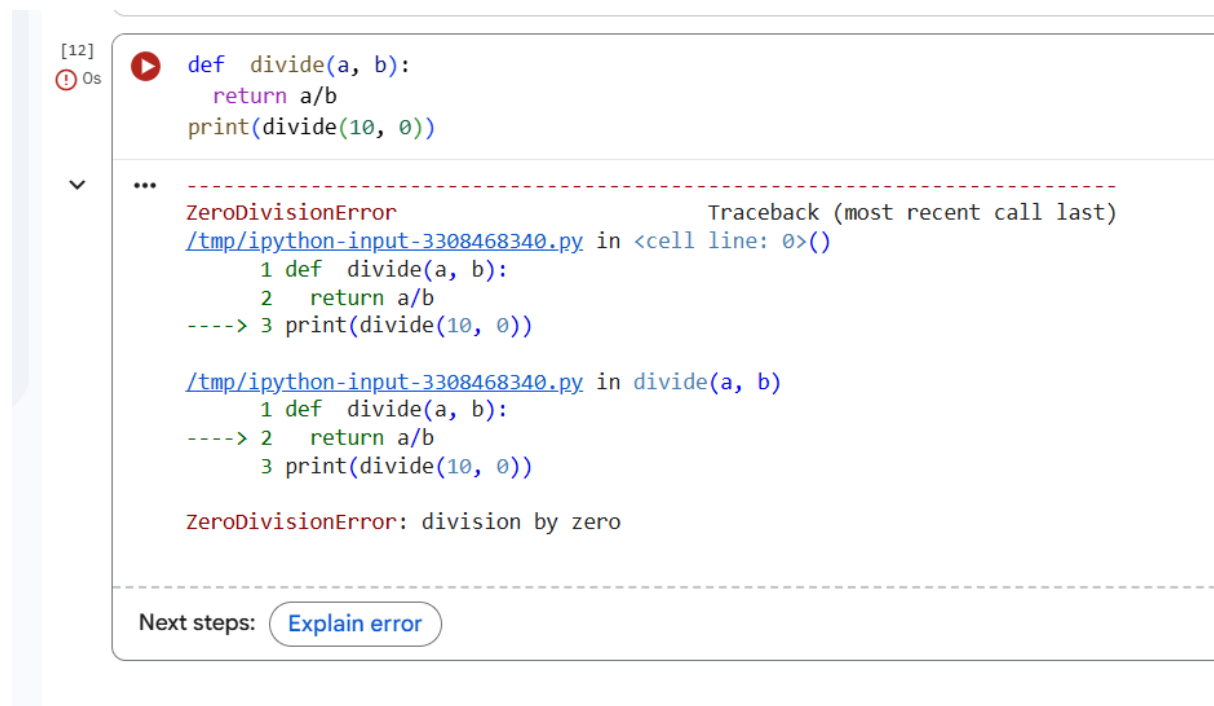
- * The loop condition depends on the variable `i`.
- * The variable `i` is not updated inside the loop body.
- * This causes the condition to remain true indefinitely.

- * As a result, the loop runs infinitely.
 - * AI identifies the missing increment statement.
 - * Adding proper increment logic fixes the infinite loop
-

TASK 3: Handling Runtime Errors (Division by Zero)

PROMPT: The following Python function crashes due to a division by zero error. Identify the runtime error and modify the code.

CODE WITH ERRORS:



The screenshot shows a Jupyter Notebook interface. On the left, a vertical sidebar contains a play button icon, a red circle with an exclamation mark, and a dropdown arrow. The main area displays the following content:

```
[12]
❗ 0s ▶ def divide(a, b):
    return a/b
    print(divide(10, 0))

...
-----
ZeroDivisionError                                Traceback (most recent call last)
/tmp/ipython-input-3308468340.py in <cell line: 0>()
      1 def divide(a, b):
      2     return a/b
----> 3 print(divide(10, 0))

/tmp/ipython-input-3308468340.py in divide(a, b)
      1 def divide(a, b):
----> 2     return a/b
      3 print(divide(10, 0))

ZeroDivisionError: division by zero
```

At the bottom, there is a section labeled "Next steps:" followed by a button labeled "Explain error".

AI FIXED CODE:

Gemini

```
[12] -def divide(a, b):  
- return a/b  
+def divide(a, b):  
+ if b == 0:  
+     raise ValueError("Cannot divide by zero.")  
+ return a / b  
print(divide(10, 0))
```

Traceback (most recent call last)
/tmp/ipython-input-3308468340.py in <cell line: 0>()
1 def divide(a, b):
2 return a/b
----> 3 print(divide(10, 0))

/tmp/ipython-input-3308468340.py in divide(a, b)
1 def divide(a, b):
2 return a/b
----> 3 print(divide(10, 0))

ZeroDivisionError: division by zero

Next steps: [Explain error](#)

Please explain this error:
ZeroDivisionError: division by zero
The error you're seeing is a `ZeroDivisionError`, which happens when you try to divide a number by zero. This is mathematically undefined and
[Accept and run](#) [Accept](#) [Cancel](#)

What can I help you build?
+ Gemini 2.5 Flash [Learn more](#)

Gemini can make mistakes, so double-check it and use code with caution. [Learn more](#)

CORRECTED CODE AND OUTPUT:

```
def divide(a,b):  
    if b == 0:  
        print("Error: Cannot divide by zero!")  
        return None  
    return a/b  
print(divide(10,0))
```

```
... Error: Cannot divide by zero!  
None
```

JUSTIFICATION:

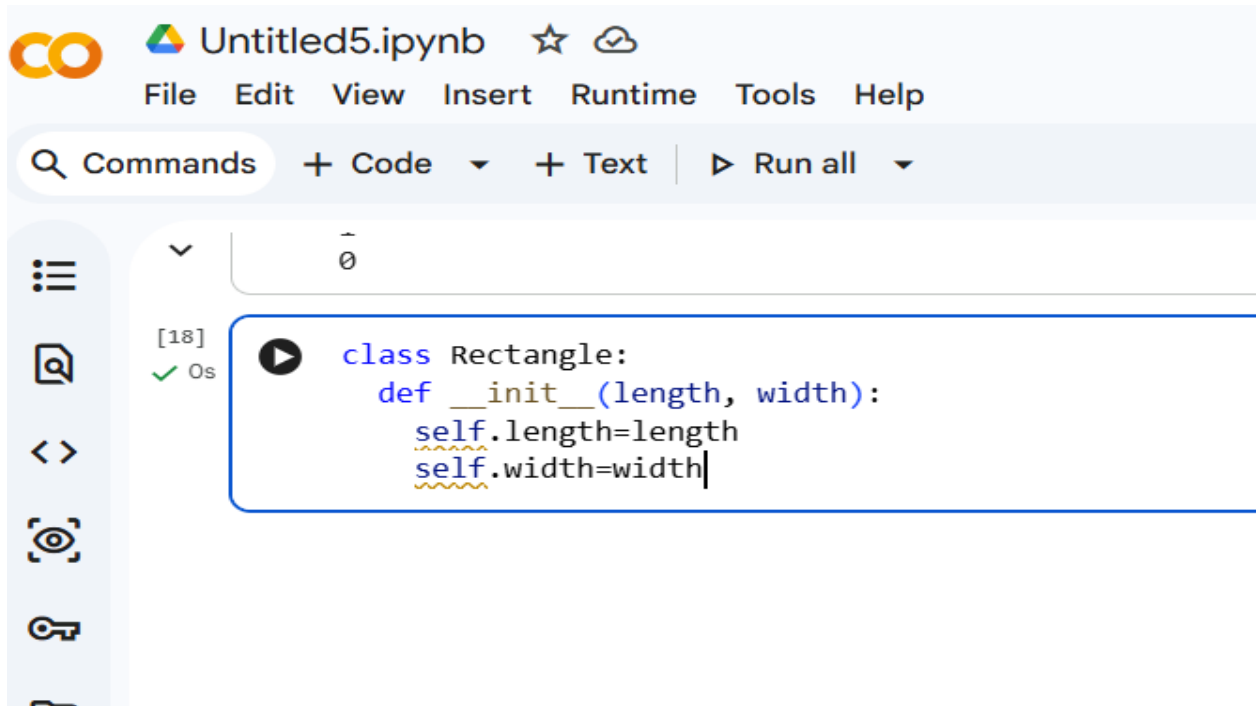
- * Dividing a number by zero causes a runtime error.
- * Python raises a `ZeroDivisionError` during execution.
- * Without handling, the program crashes immediately.
- * AI detects this risk by analyzing execution flow.
- * A try-except block prevents program termination.
- * Proper error handling ensures safe execution.

TASK4: Debugging Class Definition Errors

PROMPT:

Find and fix the error in the Python class constructor where the self parameter is missing.

CODE WITH ERRORS:

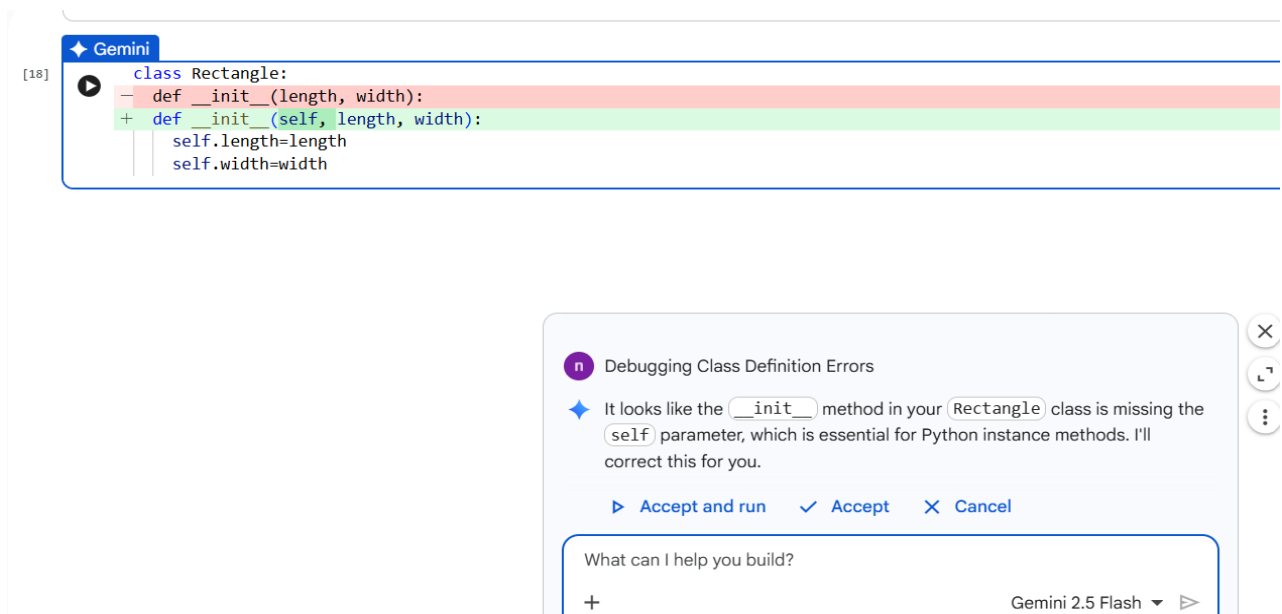


The screenshot shows a Jupyter Notebook titled "Untitled5.ipynb". The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with a search icon, "Commands", and buttons for "+ Code", "+ Text", and "Run all". On the left sidebar, there are icons for file explorer, search, expand/collapse, view, and a key icon. The main cell, labeled "[18]", contains the following Python code:

```
class Rectangle:
    def __init__(length, width):
        self.length=length
        self.width=width
```

The code is marked as successful with a green checkmark and "0s".

AI FIXED CODE:



The screenshot shows the AI's correction of the code. The code cell is labeled "[18]" and shows the original code with a red highlight on the line `def __init__(length, width):` and a green highlight on the corrected line `def __init__(self, length, width):`. The corrected code is:

```
class Rectangle:
    def __init__(self, length, width):
        self.length=length
        self.width=width
```

Below the code cell, there is a panel titled "Debugging Class Definition Errors" with a purple icon. It contains the following text:

It looks like the `__init__` method in your `Rectangle` class is missing the `self` parameter, which is essential for Python instance methods. I'll correct this for you.

At the bottom of the panel, there are three buttons: "Accept and run", "Accept", and "Cancel". Below these buttons is a text input field with the placeholder "What can I help you build?" and a "+" button. The bottom right corner of the panel shows "Gemini 2.5 Flash" and a play button.

CORRECTED CODE AND OUTPUT:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

my_rectangle = Rectangle(5, 10)
print(f"Rectangle length: {my_rectangle.length}")
print(f"Rectangle width: {my_rectangle.width}")
```

```
Rectangle length: 5
Rectangle width: 10
```

JUSTIFICATION:

- * self represents the current object in a class.
 - * It is mandatory in the `__init__()` method.
 - * Missing self prevents instance variable creation.
 - * This leads to incorrect object initialization.
 - * AI detects structural issues in class definitions.
 - * Adding self enables proper object behavior.
-

TASK 5: Resolving Index Errors in Lists.

PROMPT: Identify the `IndexError` in the following Python code. Suggest and apply safe methods such as bounds checking or exception handling to fix it.

CODE WITH ERRORS:


```
1 numbers=[1,2,3]
2 print(numbers[5])

... -----
IndexError                                Traceback (most recent call last)
/tmp/ipython-input-3809996345.py in <cell line: 0>()
      1 numbers=[1,2,3]
----> 2 print(numbers[5])

IndexError: list index out of range
```

AI FIXED CODE:

```
7] numbers=[1,2,3]
    -print(numbers[5])
    +print(numbers[0]) # Changed index to 0 to access the first element, for example.

... -----
IndexError                                Traceback (most recent call last)
/tmp/ipython-input-3809996345.py in <cell line: 0>()
      1 numbers=[1,2,3]
----> 2 print(numbers[5])

IndexError: list index out of range
```

Next steps: [Explain error](#)

CORRECT CODE AND OUTPUT:

```
[28] ✓ Os numbers=[1,2,3]
    print(numbers[0]) # Changed index to 0 to access the first element, for example.
    ... 1
```

JUSTIFICATION:

- * Python lists have fixed index boundaries.
- * Accessing an invalid index causes an `IndexError`.
- * Such errors occur at runtime.

- * AI detects index misuse by checking list size.
- * Bounds checking or exception handling prevents crashes.
- * Safe access improves program stability.